

Small Introduction To The Android SDK

Table of Contents

Introduction	1
Import the android SDK	1
Implementation	2
Getting started	2
Authentication	2
Open a book	3
Headless Player	4
SDK Player + Service	4
A more customizable player	5
Offline and Storage	5
Offline Model	5
Download Lifecycle	6
Removing Downloaded Material	6
Additional Info	6

Introduction

A brief introduction to the WeDoBooks SDK, including setup and usage. Presented as a step-by-step guide in Android Studio¹.

Import the android SDK

To set up the SDK, first add the WeDoBooks Maven repository entry to the list of repositories. In newer projects this is typically configured in settings.gradle (or settings.gradle.kts). In older projects it may be placed in the project-level build.gradle, and it can also be added in the module-level build.gradle if required. You can set up your repositories as follows:

The settings.gradle file

```
Kotlin
repositories {

    ... // Other repositories

    maven {

        url =
        uri("https://wdb-android-maven-844218222632.europe-west3.run.a
pp")

        credentials {

            // Provided by wedobooks

            username = getLocalProperty("WDB_USER_NAME")

            // Provided by wedobooks

            password = getLocalProperty("WDB_PASSWORD")

        }

    }

}
```

¹ <https://developer.android.com/studio>

The following example illustrates the module-level dependency declaration:

Kotlin

```
dependencies {  
    implement "io.wedobooks:sdk:{version_number}"  
}
```

Implementation

Getting started

This section describes how to implement the SDK. Before using the SDK, call the setup function to initialize it; this enables SDK functionality as shown in this initialization example:

The WDBSdk initialization

Kotlin

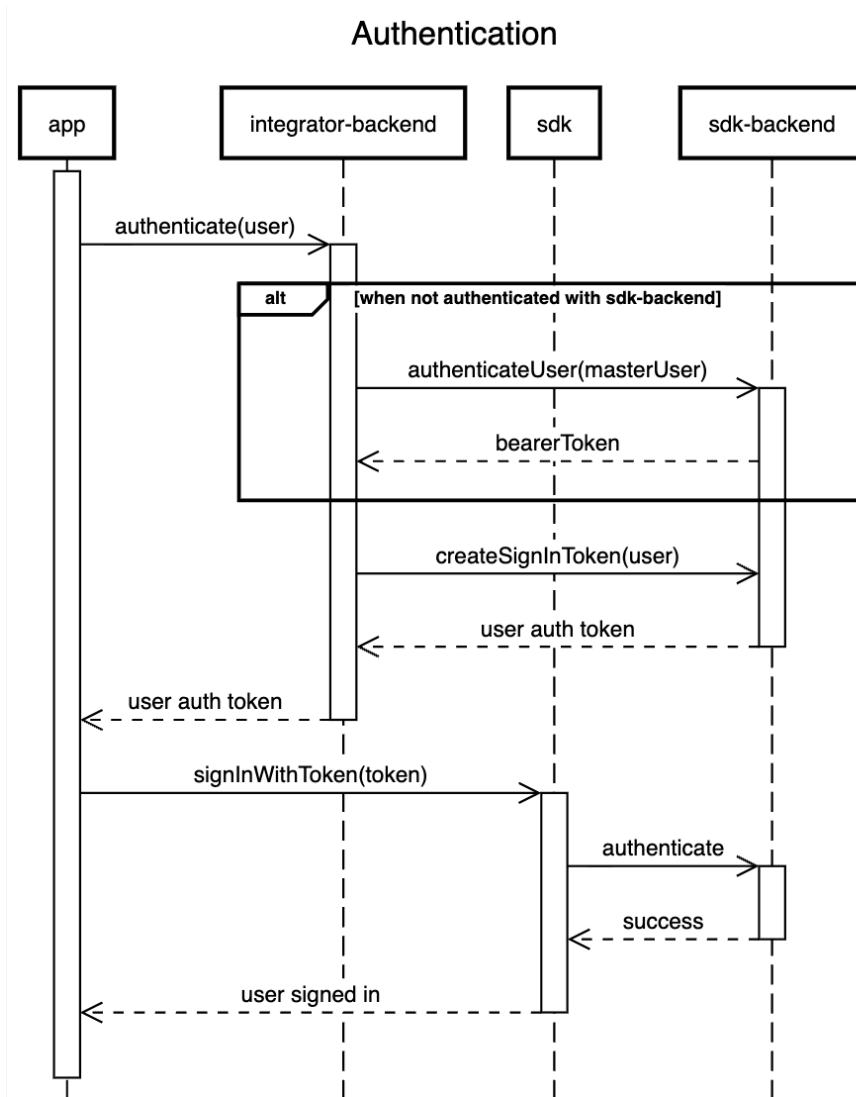
```
WeDoBooksSDK.setup(  
    context = this.applicationContext,  
    config = WeDoBooksConfiguration(  
        applicationId = BuildConfig.APPLICATION_ID,  
        firebaseApiKey = {find in firebase project},  
        firebaseProjectId = {find in firebase project},  
        firebaseAppId = {find in firebase project},  
        // Provided by WeDoBooks  
        readerApiKey = BuildConfig.READER_API_KEY,  
        readerApiSecret = BuildConfig.READER_API_SECRET  
    ),  
    themeConfig = WeDoBooksThemeConfiguration  
        .builder()  
        .build()  
)
```

The WeDoBooksThemeConfiguration shown in the setup above can be used to configure theme colors and fonts applied by the reader/player.

Authentication

Most SDK features require an authenticated user. Do note that user authentication requires a backend-to-backend integration. After the integration is in place, call `WeDoBooks.userOperations.signInWithToken`. See the authentication flow diagram below:

Authentication sequence diagram



Open a book

Provide an ISBN and call `WeDoBooksSDK.bookOperations.checkoutBook(isbn)`. The call either creates a new checkout or returns an existing one if the user already has a checkout for the given ISBN. After obtaining the checkout, the reader can be opened. The SDK automatically launches either the audio player or the e-book reader based on the ISBN.

Implementing BookScreen

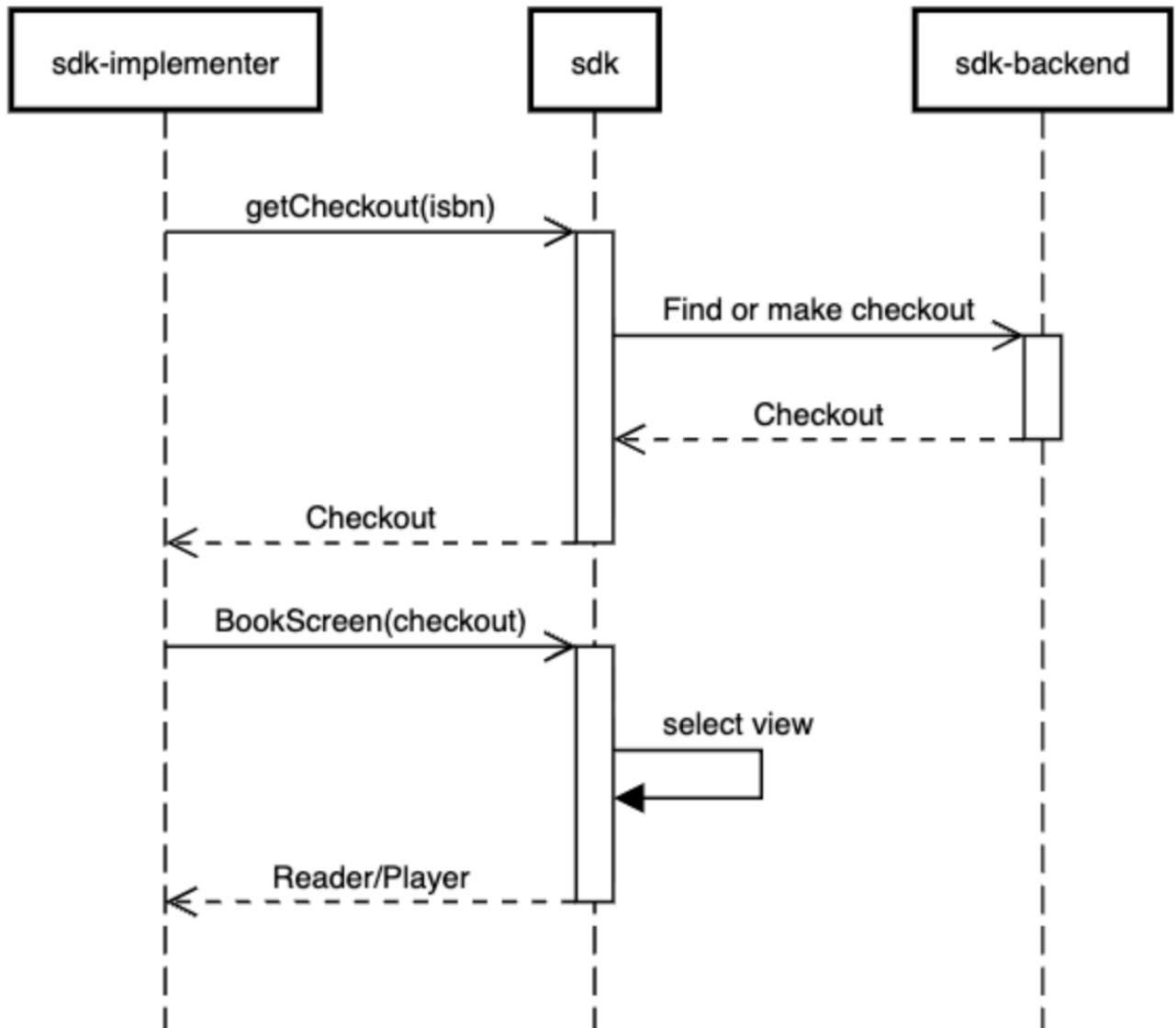
Kotlin

```
// you can set your own coverUrl else set to null if you want  
to use coverUrl provided by WeDoBooks
```

```
WeDoBooksSDK.bookOperations.BookScreen(  
    checkout = checkout,  
    cover = null,  
    // implement your own close button action  
    onCloseClick = {},  
    // there is a button when you get to the end of the ebook  
    isFinishButtonEnabled = false,  
    // action for finish button  
    onFinishClick = {},  
    // alternative close button for audioplayer  
    onAudioMinimizeClick = null,  
    // if you want to save state outside this composable  
    viewModelStoreOwner = null,  
    isDarkMode = isDarkMode  
)
```

Book opening workflow

Open a Book



Audio Player

When using the Audio player service please add the following to your manifest

None

```
<service
    android:name="io.wedobooks.player.services.AudioPlayerSessionService"
    android:exported="true"
    android:foregroundServiceType="mediaPlayback">
    <intent-filter>
        <action android:name="androidx.media3.session.MediaLibraryService"
/>
        <action android:name="android.media.browse.MediaBrowserService" />
    </intent-filter>
</service>
```

Headless Player

For audiobooks, it is possible to use a custom UI instead of the one provided by the SDK. There are two ways to achieve this:

SDK Player + Service

The SDK provides a built-in player, which is the same implementation used in the BookScreen. To integrate with this player, use `WeDoBooksSDK.headlessAudioPlayer.playerUiState`, which exposes a Flow that can be observed for UI updates. To start playback, call `WeDoBooksSDK.headlessAudioPlayer.loadAudioBook(checkout)`. This method loads and prepares the audiobook as shown here:

Load and prepare an audiobook:

Kotlin

```
// first Load the book, this throws in case it cannot load

WeDoBooksSDK.headlessAudioPlayer.LoadAudioBook(

    checkout = checkout,

    cover = null,

    initialProgressMs = null

)
```

Observe `playerUiState` to determine when the player is ready. Once ready, use `WeDoBooksSDK.headlessAudioPlayer.withAudioController` to control playback

Control playback with the audio controller

Kotlin

```
// withAudioController is how you access the controller

WeDoBooksSDK.headlessAudioPlayer.withAudioController {

    it.play()

}
```

A more customizable player

The SDK also offers a more customizable player implementation based on the Media3 Player interface. Note that this implementation does not include a foreground service by default. As a result, it is not configured for background playback or Android Auto support out of the box. An example setup for these scenarios is available in the sample application². To get started with this player, use the following builder configuration:

The Media3 Player is build

Kotlin

```
// use the builder to start

// the WdbAudioPlayer interfaces the media3 Player interface

val player: WdbAudioPlayer =
    WeDoBooksSdk.wdbAudioPlayerBuilder().build()

player.loadBook(checkout)
```

² <https://github.com/wedobooks/wedobooks-sdk-android-sample>

Offline and Storage

Offline Model

Downloaded material is scoped to the current signed-in user. The SDK stores downloaded ebook and audio artifacts in app storage, and stores ebook offline keys in the datastore. Ebooks are downloaded before they are opened, so they are available offline from that point until the material is explicitly removed through the API. Audiobooks are downloaded through a call to the API. Once a download completes successfully, the book can be opened while offline.

Download Lifecycle

A download begins by acquiring a checkout, which can be obtained through `WeDoBooksSDK.bookOperations.checkoutBook(isbn)` or `WeDoBooksSDK.bookOperations.allCheckoutsFlow`. The checkout is then passed to `WeDoBooksSDK.storageOperations.downloadBook(checkout)` as demonstrated in the following function. Downloaded material is opened the same way as a book that has not been downloaded. Implement book downloads

Download a book

Kotlin

```
suspend fun downloadBook() {  
    val checkout =  
        WeDoBooksSdk.bookOperations.checkoutBook(isbn)  
    try {  
        WeDoBooksSdk.storageOperations.downloadBook(checkout)  
    } catch (e: Throwable) {  
        Log.d("error", e?.message.orEmpty())  
    }  
}
```

Track download progress using the flow

Kotlin

```
val isbn = checkout.materialId

WeDoBooksSdk.storageOperations.bookDownloadsFlow

    .map { it[isbn] }

    .filter { it != null }

    .onEach {

        Log.d("progress", " ${it?.progress ?: 0}")

    }

    .launchIn(viewModelScope)
```

The flow also reports the current status of the book, including whether the download failed.

Removing Downloaded Material

Books are removed using `WeDoBooksSDK.storageOperations.removeBook(isbn)` or `WeDoBooksSDK.storageOperations.removeAll()`. Upon successful removal, the key for that book is no longer present in `bookDownloadsFlow`.

Additional Info

The SDK provides more than obtaining book checkouts and launching the reader. For concrete examples, refer to the sample application³, which demonstrates user reading statistics and the implementation of a “currently playing” bar.

For additional customization options and integration points, consult the SDK API documentation⁴. The API exposes extra endpoints and configuration for checkouts and reading statistics.

³ <https://github.com/wedobooks/wedobooks-sdk-android-sample>

⁴ <https://sdk-stage-bokus-api-gateway-7x7f3u67.nw.gateway.dev/docs>